HATHOR-CORE · MASTER-BOOK

The Life of the Node — Everything Working Together

One worked example — Alice pays Bob 30 HTR — followed through every subsystem, from the connection that carries it to the block that confirms it.

SUBJECT	hathor-core · Capstone
CHAPTER	44 · Capstone
BRANCH	study/hathor-core-studies
EDITION	2026 · v1

End-to-end · Life of a connection · Life of a transaction · Life of a block · Nano-contract call · Synthesis · Alice pays Bob

Table of Contents

Chapter 44 — The Life of the Node: Everything Working Together

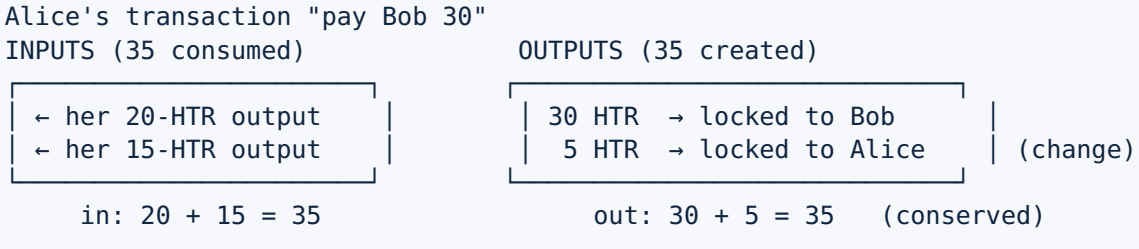
What this chapter does

- Takes one concrete event — **Alice sends Bob 30 HTR** — and follows it through every subsystem the book has described, so the machine is finally seen running as a whole.
- Traces four lifecycles in turn: a **connection** (the stage everything happens on), a **transaction** (the spine), a **block** (told as the differences from the transaction), and a **nano-contract call** (the one path that breaks the usual rules).
- Re-introduces nothing. Each step says what happens here and why, then points to the chapter that explains it in depth (`→ Ch N`). If a step is unclear, that chapter is the place to go.

Every chapter until now took one subsystem and opened it up. This one does the reverse: it keeps a single event moving and lets you watch each subsystem do its part as the event passes through. The goal is the thread, not the detail — you have the detail already. A developer who reads only this chapter should come away understanding how the pieces connect; a developer who wants to know why any single step works the way it does has forty-three chapters waiting.

We anchor everything on one scenario, fixed now and carried to the end:

The scenario. Alice's wallet holds two unspent outputs locked to her address: one worth **20 HTR** and one worth **15 HTR** (35 total). She wants to pay **Bob 30 HTR**. Her node is `N_A`; Bob runs `N_B`; between and around them is the rest of the network. We follow the payment from the moment her wallet decides to spend, to the moment a block buries it as settled history.



Part 1 — The Life of a Connection

Before any payment can move, a node must be part of the network. The connection is the stage; the transaction is the actor that walks onto it. We follow how `N_A` comes to have a live, synced link to a peer.

1.1 Boot and assembly

`N_A` starts when an operator runs `hathor-cli run_node` (→ Ch 21). The command is dispatched, arguments are parsed, and the settings profile for the network is loaded and frozen — this fixes the genesis, the timing constants, and every rule the node will enforce, so that `N_A` speaks the same dialect as every other node (→ Ch 22). The Twisted reactor — the single event loop the whole node will run on — is created but not yet started (→ Ch 16, 23).

The builder then constructs and wires every subsystem into one `HathorManager`: storage opens, indexes are prepared, the wallet loads, the pub-sub bus and the verification and consensus services come up (→ Ch 24). The manager's `start()` runs them in order — crash-check, load genesis, narrow the storage scope to valid data, bring each subsystem online — and marks the node `READY`. The very last act of boot hands the one thread to the reactor, and from here `N_A` is reactive: asleep until something happens (→ Ch 29).

1.2 Finding a peer and shaking hands

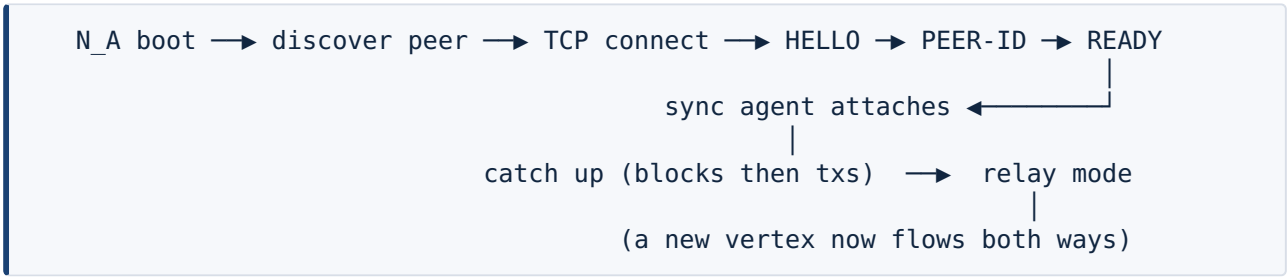
A freshly-started node knows no peers. The connections manager discovers them — from a bootstrap list and from peers that share the addresses they know — and opens an outbound TCP connection to one (→ Ch 34). Each connection is driven by its own protocol object running on the reactor, and it climbs a three-step handshake state machine. In **HELLO**, the two nodes exchange protocol and sync versions, confirm they are on the same network, and agree on the genesis. In **PEER-ID**, each proves its identity. Only when both sides are satisfied does the connection reach **READY** (→ Ch 34).

The handshake is not ceremony. The network match prevents a testnet node from polluting a mainnet peer; the identity step lets a node recognise and de-duplicate peers; the version negotiation picks the one sync protocol both speak. A connection that fails any step is dropped, which is a normal outcome rather than an error.

1.3 Catching up: sync

The moment a connection is READY, a **sync agent** attaches to it and the two nodes reconcile their ledgers (→ Ch 35). If `N_A` is behind, the agent finds the highest block the two share — an n-ary search over block heights — then streams the missing blocks in height order, and for each block streams the transactions it confirms, handing every received vertex into the ingestion pipeline we are about to follow in Part 2. Once caught up, the agent flips into **relay** mode: from now on, any new vertex either node learns about is pushed to the other in real time.

That relay subscription is the bridge between this part and the next. When Alice's wallet finally broadcasts her payment, it travels to `N_B` — and to the wider network — over exactly these standing, synced connections.



Part 2 — The Life of a Transaction

Now the actor walks on. We follow Alice's payment from her wallet's decision to spend, through verification and consensus, into storage, and back out as a notification she can see. This is the spine of the whole node, and the longest trace.

2.1 The wallet builds the transaction

Alice's wallet has been tracking which outputs she can spend by watching the ledger and keeping her unspent outputs in view (→ Ch 40, 28). When she asks to send 30 HTR, the wallet performs input selection: it picks a set of her unspent outputs whose values cover 30, here the 20-HTR and the 15-HTR outputs, totalling 35. Because outputs are spent whole and 35 exceeds 30, the wallet creates a second output — 5 HTR back to Alice — as change (→ Ch 7).

The transaction now has a definite shape: two **inputs**, each a pointer to one of Alice's prior outputs, and two **outputs**, one of 30 locked to Bob's address and one of 5 locked back to Alice. The wallet also chooses two **parents** for the transaction — tips of the DAG it will confirm — because in Hathor a transaction attaches itself directly to the graph rather than waiting to be packed into a block (→ Ch 8). At this point the transaction is structurally complete but unsigned and unproven.

2.2 Signing: proving ownership

Each of Alice's two inputs spends an output that was locked to her address with a script demanding a signature from her key (→ Ch 7, 31). The wallet computes the transaction's sighash — a hash over the transaction's contents that the signature will commit to — and signs it with Alice's private key, producing, for each input, an unlocking payload of `<signature> <public-key>` (→ Ch 40). This is the heart of ownership in a UTXO ledger: Alice does not assert that she owns the coins, she demonstrates it by producing data that satisfies each output's lock. No name, no account — a satisfiable lock and the key to open it.

2.3 Proof-of-work: the anti-spam cost

A Hathor transaction, not only a block, carries proof-of-work (→ Ch 9). Before it can be accepted anywhere, Alice's transaction must meet a minimum weight computed from its size and amount — a small cost that makes flooding the DAG with junk transactions expensive. The wallet grinds a nonce until the transaction's hash falls under the target implied by that weight (→ Ch 37). Because this is a brief burst of pure computation, it

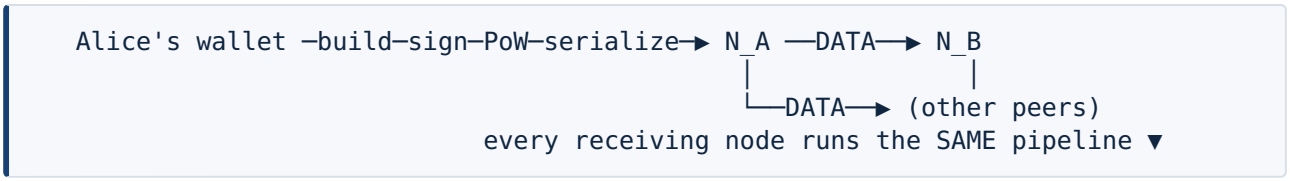
runs off the reactor in a thread pool, so the node it is attached to never freezes while the work is done (→ Ch 2, 16). When a valid nonce is found, the transaction has its identity — its hash — and is ready to enter the world.

2.4 Serialization: becoming bytes

To leave Alice's wallet, the transaction must become a flat sequence of bytes — and the exact bytes matter, because the hash everyone will use to identify and verify it is computed over them (→ Ch 26). Hathor's bespoke binary format encodes the version and signal bits, then the funds section (the token list, inputs, and outputs), then the graph section (the parents), then the nonce, in a fixed, deterministic order. Determinism is the point: every node that serializes this transaction produces byte-for-byte the same result, so every node computes the same hash and reaches the same verdict. The same bytes will be sent on the wire and written to disk.

2.5 Propagation: onto the network

Alice's node hands the serialized transaction to its peers over the standing relay connections from Part 1 (→ Ch 34). It travels as one line on the wire — a `DATA` message carrying the transaction's bytes — to each connected peer, including, after a hop or two across the network, Bob's node `N_B`. Each peer that receives it will run the very same acceptance pipeline that `N_A` ran when it first built or received the transaction; there is no privileged path. From here we follow the transaction's arrival at one node — say `N_B` — knowing every other node does the identical thing.



2.6 Ingestion: the vertex handler

When the bytes arrive at `N_B`, they are parsed back into a transaction object and handed to the **vertex handler** — the single chokepoint where "a vertex arrived" becomes "the ledger changed" (→ Ch 33). The handler first checks whether the transaction is already known, and whether its dependencies — the outputs it spends and the parents it names — are present. If everything it needs is in hand, the handler orchestrates the rest of this part as one ordered sequence: verify, then reach consensus, then save and announce. If a dependency is missing, the transaction waits until sync supplies it, so a vertex is never processed before the things it rests on.

2.7 Verification: is it valid on its own?

The verifier asks a single question: is this transaction well-formed and rule-abiding in isolation, regardless of anything else on the ledger (→ Ch 31)? It re-computes the hash and confirms the proof-of-work meets the target. It checks the structure and weight. For each input, it runs the unlocking script against the locked output's script on a small stack machine — this is where Alice's signatures are actually checked against the public keys, and a forged or mismatched signature fails here (→ Ch 31). It confirms conservation: the inputs sum to 35, the outputs sum to 35, nothing is created from nothing. A transaction that fails any check is rejected outright and never enters the ledger. Alice's is well-formed, correctly signed, and balanced, so it passes.

2.8 Consensus: which history does it belong to?

Verification proved the transaction is valid; consensus decides whether it is canonical (→ Ch 32). The key question for a payment is conflict: does any other transaction already in the DAG spend one of the same two outputs Alice is spending? If not — the normal case — Alice's transaction is accepted as executed, its metadata records it as not voided, and the outputs it consumes are now marked spent. If a conflicting transaction did exist (a double-spend), consensus would compare the accumulated weight behind each and let the heavier one win, voiding the lighter; on an exact tie, both stay voided until a later vertex breaks the deadlock (→ Ch 32). Because Alice spent her coins once, honestly, her transaction takes its place in the canonical ledger without contest.

2.9 Storage: writing it down

With the transaction verified and its consensus state decided, the node persists it (→ Ch 27). The serialized body is written under the transaction's hash in one RocksDB column family; its mutable metadata — the consensus verdict, the accumulated weight, the spent markers — goes in another; its immutable computed metadata in a third. The two outputs Alice spent are not deleted; they are recorded as spent by this transaction, which is what lets a later reorganization cleanly reverse the spend if consensus ever changes its mind. The transaction is now durable: a restart of the node will find it exactly here.

2.10 Indexes: making it findable

Storing the transaction by hash answers "give me the vertex with this hash," but not the questions wallets actually ask — "what can Bob spend now?" The node updates its derived indexes to reflect the new reality (→ Ch 28). The UTXO index removes Alice's two consumed outputs and adds the two new ones: a 30-HTR output now spendable by Bob, and a 5-HTR change output spendable by Alice. The address index records that

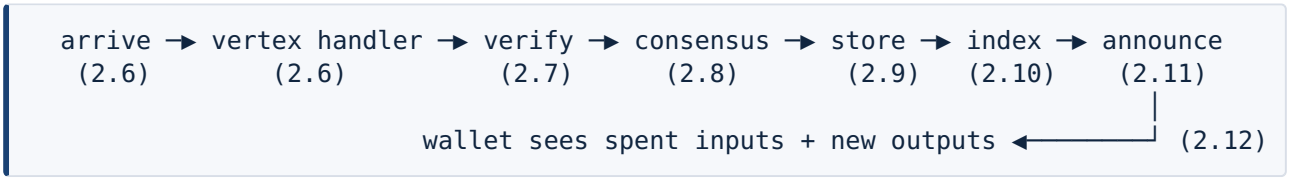
Bob's and Alice's addresses each have new history. None of this is new truth — every index entry could be rebuilt from the stored transactions — but it turns an expensive scan into an instant lookup.

2.11 Announcement: telling the interested

Finally the node announces what happened on its internal pub-sub bus (→ Ch 30). Components that subscribed to "a new transaction was accepted" react: the metrics counters tick, any connected wallet or dashboard watching over a WebSocket is notified (→ Ch 36, 42), and — if the durable event queue is enabled — the event is appended to a replayable log for downstream systems (→ Ch 30). The same acceptance also re-enters the relay path: `N_B` now forwards the transaction to its peers, which is how the payment ripples out until the whole network holds it.

2.12 The wallet sees its change

The loop closes back at Alice. Her wallet is subscribed to the same kind of notification, and when her node accepts the transaction, the wallet sees the two events that concern her: her 20-HTR and 15-HTR outputs are now spent, and a fresh 5-HTR change output is now hers (→ Ch 40). Bob's wallet, on his node, likewise sees a new 30-HTR output it controls. Neither wallet stores a balance; each re-sums its unspent outputs, and the numbers have moved by 30. The payment has happened — but, as the next part shows, it is not yet final.



Part 3 — The Life of a Block

A block travels much of the same road as a transaction — it is also a vertex, also verified, also run through consensus and stored. So rather than repeat Part 2, this part tells only the **differences**: what a block does that a transaction does not. The block we follow is the one that will eventually confirm Alice's payment and turn it from "accepted" into "settled."

3.1 The template: assembling a candidate

A transaction is built by a wallet; a block is built by the node, on request, as a template for miners (→ Ch 37). The node selects the parents the block will extend — including the previous block, which keeps the block backbone intact — sets the reward outputs that mint new HTR, and computes the minimum weight the block must meet, tuned by the difficulty algorithm toward the network's target of one block roughly every 30 seconds (→ Ch 9). The template is a fill-in-the-nonce puzzle: everything is fixed except the number a miner must find.

3.2 Mining and submission

Unlike Alice's transaction, whose small proof-of-work her wallet did itself, a block's proof-of-work is large and is done by external miners — real mining hardware speaking the Stratum protocol, or a merged-mining setup riding on Bitcoin (→ Ch 37). A miner repeatedly hashes the template with different nonces until one produces a hash under the block's hard target. When it succeeds, it submits the solved block back to the node, which feeds it into the same ingestion pipeline Alice's transaction took — vertex handler, verification, consensus, storage — with the block-specific rules layered on.

3.3 What consensus does differently for a block

Here is the real divergence. A transaction's consensus question is "does it conflict with another spend?"; a block's is "does it change which chain is the canonical one?" (→ Ch 32). Blocks compete by score — the total work of the sub-DAG behind them — and the chain with the highest score is the real history. When the new block extends the current best chain, it becomes the new tip with nothing more to decide. But if a competing branch had quietly grown heavier, accepting this block triggers a **reorganization**: the node switches to the heavier branch, voids the blocks and transactions unique to the branch it abandons, and re-applies any transactions that belong on the new one (→ Ch 32). This is the mechanism behind "finality is only probabilistic" — a recently-accepted transaction can still be undone by a reorg, which is why depth matters.

3.4 Confirming Alice's payment

The block we are following does not conflict with anything; it extends the best chain, and in doing so it reaches back through its parents to confirm the region of the DAG that contains Alice's transaction (→ Ch 8, 9). That confirmation is what changes Alice's payment from "accepted by the network" to "buried under work." Each further block built on top adds more accumulated weight that an attacker would have to out-compute to reverse her payment, so the probability of reversal falls steeply with every block (→ Ch 9). After a handful of blocks, Bob can treat the 30 HTR as his to spend without practical risk.

3.5 Two side-effects only blocks have

Accepting a block also advances two machineries a transaction never touches. First, **feature activation**: the block's signal bits are counted into the rolling tally that decides whether a proposed protocol upgrade has gathered enough miner support to lock in and activate (→ Ch 38). Second, **nano-contract execution**: contracts do not run when their calling transaction arrives — they run now, at block consensus, in a deterministic order, with their results committed to contract state (→ Ch 39, and Part 4 below). A block, then, is not only a confirmation of past transactions but the heartbeat that drives the network's slow-moving machinery forward.

```
Transaction trace (Part 2):  build — verify — consensus(conflict?) — store — announce
Block trace adds/changes:    template — EXTERNAL mining — consensus(SCORE, reorg?)
                              — reward minting — confirms txs beneath it
                              — feature signalling — runs nano-contracts
```

Part 4 — The Life of a Nano-Contract Call

One last trace, because it breaks an assumption that held through Parts 2 and 3: that a vertex's effect is settled the moment consensus accepts it. A nano-contract call is a transaction, and it travels the entire Part 2 pipeline — built, signed, serialized, propagated, verified, consensus-checked, stored. But what the contract does happens later, and elsewhere.

4.1 A call arrives, but does not run

When a nano-contract transaction is accepted, its method does not execute on arrival (→ Ch 39). The transaction carries, in a header, the identity of the contract and the method to call, plus any token deposits or withdrawals the call makes. Acceptance records the intent to run, but execution is deferred — because running a state-changing program the instant it arrives, before the network has agreed on its place in history, would let conflicting or soon-to-be-voided calls corrupt contract state.

4.2 Execution at block consensus

Instead, contracts run when a **block** confirms them, as part of that block's consensus (→ Ch 39, 32). The block fixes an order, and the calls it confirms are executed in a deterministic, seeded sequence so that every node, replaying the same block, produces the same result. Each call is handed to the runner, which executes the blueprint's method against the contract's current state. A resource-metering layer is intended to bound how much work a call may do — though on the current branch that bound is scaffolded rather than enforced, while the sandbox that restricts what a contract may touch is active (→ Ch 39). This is the one place in the node where the "execute on arrival" model of Part 2 is deliberately set aside.

4.3 State, and the cost of failure

A contract's state lives not in outputs but in a verifiable Merkle/Patricia trie, one per contract, whose root is anchored into the confirming block (→ Ch 39). A successful call commits its state changes into the trie and moves the contract forward. A call that fails — runs out of its (intended) budget, violates a rule, raises — is marked with a dedicated voiding identifier, its state changes are rolled back, and the failure is recorded rather than allowed to corrupt anything (→ Ch 32, 39). Because state is trie-

backed and block-anchored, two nodes can prove to each other that they hold the same contract state by comparing roots — the same verifiability that the rest of the ledger gets from its hashes.

Recap

Lifecycle	Spans	The one thing to remember
Connection (Part 1)	Ch 21–24, 29, 34, 35	a synced, relayed link is the stage every vertex travels on
Transaction (Part 2)	Ch 7, 8, 9, 26, 28, 30–34, 37, 40	arrive → verify → consensus → store → index → announce, at every node identically
Block (Part 3)	Ch 9, 32, 37, 38, 39	same pipeline, but consensus is by score, and a block confirms and triggers
Nano-contract (Part 4)	Ch 32, 39	accepted on arrival, but executed later, at block consensus, against trie state

Trace one payment all the way through and the node stops being a collection of packages and becomes a single machine: a wallet that proves ownership with a signature, a binary format that makes the proof reproducible, a network that carries it, a pipeline that checks and records it, a block that buries it, and a chain of further blocks that makes burying it irreversible. Every subsystem in this book exists to play one part in that sequence, and the sequence is the node. With the four lifecycles in hand — and the chapters behind each step when a detail is wanted — you can now open `hathor-core` anywhere and know not just what a piece does, but where it sits in the life of the thing passing through it.